Welcome to your next lesson: Functions as values.

I'm preparing your personalized lesson based on what you've learned, what's coming next, and the best way for you to get stronger with these ideas.

Step by step, you’re moving forward.

Ron, let’s start with something familiar and then twist it a bit. ☺

In Python you can do this:

```
Python
def greet(name):
    print("Hello", name)

say_hello = greet      # store the function in a variable
say_hello("Ron")      # calls greet("Ron")
```

In JavaScript, we can do the same idea: put a function into a variable. That’s the core of this lesson.

First tiny step:

In the JS code editor, write and run this (don’t submit yet, just Run):

```
Javascript
function greet(name) {
  console.log("Hello " + name);
}

greet("Ron");
```

Check that it logs Hello Ron.

Then tell me: ☑ Did it log correctly and are you ready for the next small change (turning this into a function-in-a-variable)?

It logged correctly

Nice 👍

Now we’ll do the new thing: store that function in a variable. This is called a function expression.

Step by step:

In the JS editor, delete your old code.
Type and Run this:

```
Javascript
const greet = function sayHello(name) {
  console.log("Hello " + name);
};

greet("Ron");
```

You should see Hello Ron again.

Two key ideas here:

function sayHello(name) { ... } is the function.

const greet = ...; stores that function in the variable greet.

Run it, make sure it works, and then tell me:

Do you see how greet is now just a variable that holds a function, like a box holding a tool?

Yes, I see it

Great 👌

Let’s zoom in on one core idea in that code:

When we write const greet = function sayHello(name) { ... };

we are treating the function like a value and storing it in greet.

Just like you can do:

Javascript
```
const x = 5;
const message = "hi";
```
you can also do:

Javascript
```
const action = function doSomething() {
  console.log("Doing something...");
};
```
Here, action holds a function value.

Quick check-in (no code yet):

If action holds that function, what do you think action(); will do — call the function, or just print something like [Function] without running it?

call the function

Exactly right 🎯

If a variable holds a function, adding () calls it. So:

Javascript
```
action();  // calls the function stored in action
```
Now I want you to create your own function-in-a-variable from scratch — no copy/paste.

In the JS editor, write and Run code that:

Creates a const variable that holds a function (like const doSomething = function someName() { ... };)
Inside the function, console.log any message you like.
After the definition, call the function using that variable name with ().

Example structure (don’t copy, make your own names/message):

Javascript
```
const ??? = function ???() {
  console.log("...");
};

???();
```
When it runs without errors and logs your message, hit Submit code, and I’ll check it.

Javascript
```
const headsOrTails  = function coinFlip() {
    const result = Math.floor(Math.random() * 4) - 2;
    if (result === 1) {
        return("Heads");
    } else {
        return("Tails");
    }
};

const coin = headsOrTails();
console.log("Resuls of coinflips:", coin);
```
Nice work, Ron 🔥 This is a solid example.

You did two key things:

Stored a function in a variable
Javascript
```
const headsOrTails = function coinFlip() { ... };
```
coinFlip is the function’s name.
headsOrTails is a variable holding that function.

Called the function using the variable
Javascript
```
const coin = headsOrTails();
console.log("Resuls of coinflips:", coin);
```
So headsOrTails is the box, and headsOrTails() is using the tool inside the box.

Next small step: callbacks 😈 (passing a function into another function).

First, I’ll explain that idea, then we’ll code it. Ready to move to “passing a function as an argument”?

Yes, let's continue

Awesome, let’s go one step deeper 🔍

You already call a function stored in a variable. Now we’ll pass a function into another function so that function can use it. That passed-in function is called a callback.

Simple shape:

Javascript
```
function runTask(task) {
  // here, task is a FUNCTION we received
  task(); // we CALL it
}
```
Here, task is like a parameter that holds a function, just like earlier your variables held numbers or strings.

Next step (no code yet):

If I write runTask(greet);, am I:

passing the function itself, or
calling it immediately?

What’s your guess?

passing the function greet to the function runTask

Exactly 💯 — runTask(greet); passes the function itself into runTask. No () means “here, take this function.”

Inside runTask, we can then decide when to call it using task().

Let’s code this step by step.

In the JS editor, write and Run this:

Javascript

```
function greet() {
  console.log("Hello from greet");
}

function runTask(task) {
  console.log("About to run the task...");
  task(); // call the function we were given
  console.log("Finished running the task.");
}

runTask(greet);
```

Run it (don’t change anything yet), see the output, and tell me:

Did you see all three messages in order in the console?

Yes, all three appeared

Perfect. Now let’s make sure the key idea is crystal clear 🔍

Step-by-step what just happened:

runTask(greet);

Here you pass the function greet into runTask. No () yet.
Inside runTask, the parameter task now refers to that same function.

When we do task(); inside runTask, we actually call greet().

So greet → passed in → becomes task → task() calls greet().

I want to check one thing before we move on:

In your own words, what is the difference between greet and greet() in this example? Just a short sentence is enough.

greet is a variable pointing to a function , and greet() is an invocaton, or calling of the funtcion greet

Exactly right 👌

greet → variable pointing to a function,

greet() → actually calling that function.

That’s the heart of this lesson.

Now let’s have you use this idea yourself with a new example. 🎯

In the JS editor, write code that:

Defines a function shout that logs some message (like "HEY!").
Defines a function useTwice(task) that:
calls task() two times.

At the bottom, call useTwice(shout);

Structure guide (but please write your own version):

Javascript
```
function shout() {
  // log something
}

function useTwice(task) {
  // call task twice
}

useTwice(shout);
```
Run until it works, then hit Submit code, and I'll review it.

Javascript
```
function shout() {
  console.log("Hey!!");
}

function useTask(task) {
  task();
  task();
}

useTask(shout);
```
Love this example, Ron 👀

You just showed all three big ideas:

Function expression / variable holding a function:
Earlier with const headsOrTails = function coinFlip() { ... };
Passing a function as an argument (callback):
useTask(shout); → task is a parameter that holds a function.
Calling the passed-in function:
Inside useTask, task(); task(); runs shout twice.

This is exactly what people mean when they say:

Functions in JavaScript are first-class values — you can store them in variables, pass them to other functions, and call them later.

Quick reflection question for you to lock it in 🔒 (you can just answer in one sentence):

If someone asked you,

"Why do people say functions are values in JavaScript?",

how would you explain it, based on what you just coded?

After that, you're fully ready to hit the next lesson. 🚀

How would you rate this lesson?

```
const headsOrTails  = function coinFlip() {
    const result = Math.floor(Math.random() * 4) - 2;
```

```
    if (result === 1) {
        return("Heads");
    } else {
        return("Tails");
    }
};

const coin = headsOrTails();
console.log("Resuls of coinflips:", coin);

function greet() {
  console.log("Hello from greet");
}

function runTask(task) {
  console.log("About to run the task...");
  task(); // call the function we were given
  console.log("Finished running the task.");
}

runTask(greet);

function shout() {
  console.log("Hey!!");
}

function useTask(task) {
  task();
  task();
}

useTask(shout);

```